# Lecture 5. Linear Regression

Simple linear regression, loss functions, multiple regression, model selection, interactions

## Contents


**5.1 Simple linear regression** **2**
- 5.1.1 Example: the rail trail . . . 2
- 5.1.2 Prediction and intervals . . . 3
- 5.1.3 Strength of fit and least squares . . . 4
- 5.1.4 A categorical predictor . . . 4

**5.2 Loss functions** **5**

**5.3 Multiple regression** **6**
- 5.3.1 Inference for coefficients . . . 6
- 5.3.2 Choosing a model . . . 7

**5.4 Other considerations** **8**

## 5.1 Simple linear regression

**Motivation.** The most common question in statistics is whether a variable $Y$ is associated with predictors $X_1, \ldots, X_p$, how, and whether the association can be used to predict $Y$. Linear regression is the simplest complete answer, and its ideas (a loss, a fitted family, uncertainty of the fit) carry over to every later model.

Two related tasks: **estimation** uses data to determine model parameters; **prediction** uses the fitted model to predict outputs for unseen data. We call $y$ the *outcome* (response) and $x$ the *predictor* (explanatory variable), avoiding "dependent" and "independent", which have other meanings in statistics.

The simple linear regression model is

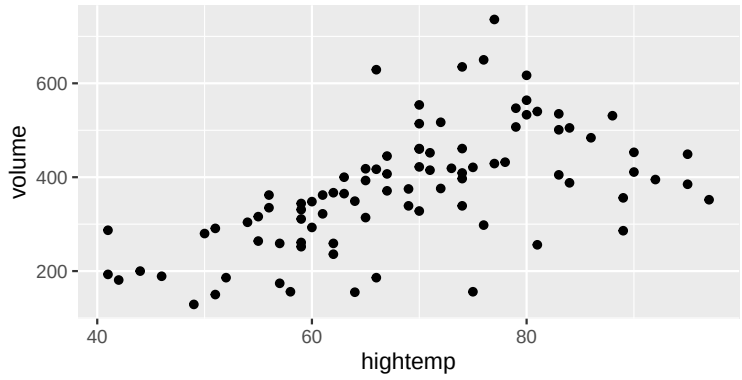
$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \qquad i = 1, \ldots, n,$$

with population intercept $\beta_0$, slope $\beta_1$ and random errors $\epsilon_i$ with mean 0. The coefficients are unknown; `lm()` finds the best estimates $\hat{\beta}_0$, $\hat{\beta}_1$, giving fitted values $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$ and residuals $\hat{\epsilon}_i = y_i - \hat{y}_i$.

### 5.1.1 Example: the rail trail

`mosaicData::RailTrail` records daily ridership (`volume`, riders and walkers on a bike path in Florence, Massachusetts) over 90 days, with temperature, rainfall, cloud cover and day of the week. The planning commission wants to understand how ridership relates to these variables.

```
ggplot(RailTrail, aes(x = hightemp, y = volume)) + geom_point()
```




```
mod <- lm(volume ~ hightemp, data = RailTrail)
tidy(mod)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic       p.value
##   <chr>          <dbl>     <dbl>     <dbl>         <dbl>
## 1 (Intercept)   -17.1     59.4      -0.288 0.774
## 2 hightemp        5.70     0.848     6.72  0.00000000171
```

$\hat{\beta}_1 = 5.70$: about 5.7 more trail users are predicted for each additional degree (Fahrenheit) of high temperature. The intercept, the predicted volume at 0°F, is far outside the data and not interesting. `summary(mod)` adds the residual standard error, $R^2$ and the overall $F$-test:

```
summary(mod)
```

```
##
## Call:
```

```
## lm(formula = volume ~ hightemp, data = RailTrail)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -254.562  -57.800    8.737   57.352  314.035
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -17.079     59.395  -0.288    0.774
## hightemp       5.702      0.848   6.724 1.71e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 104.2 on 88 degrees of freedom
## Multiple R-squared:  0.3394, Adjusted R-squared:  0.3319
## F-statistic: 45.21 on 1 and 88 DF,  p-value: 1.705e-09
```

### 5.1.2 Prediction and intervals

```
predict(mod, newdata = data.frame(hightemp = c(30, 40)))
```

```
##        1        2
## 153.9771 210.9958
```

```
predict(mod, interval = "confidence")[1:3, ]
```

```
##        fit      lwr      upr
## 1 456.1766 423.8301 488.5231
## 2 399.1578 376.2313 422.0843
## 3 404.8597 381.3622 428.3572
```

```
predict(mod, interval = "prediction")[1:3, ]
```

```
## Warning in predict.lm(mod, interval = "prediction"): predictions on current data refer to
    _future_ responses
```

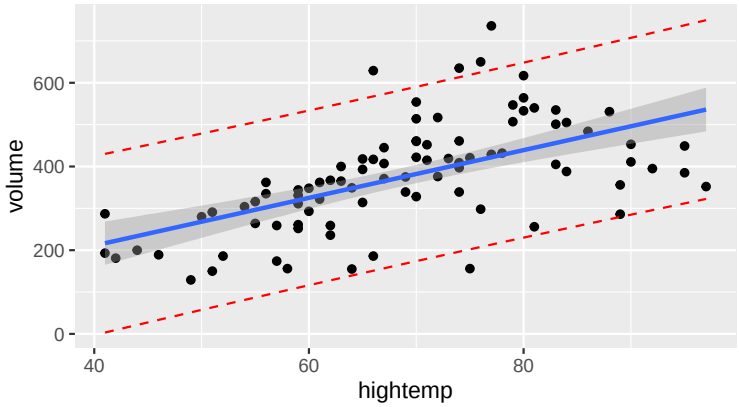
```
##        fit      lwr      upr
## 1 456.1766 246.6175 665.7357
## 2 399.1578 190.8447 607.4709
## 3 404.8597 196.4830 613.2364
```

The **confidence interval** is for the mean volume at that temperature; the **prediction interval** is for one new day, and is much wider because it adds the day-to-day noise $\epsilon$.

```
pi <- predict(mod, interval = "prediction")
```

```
## Warning in predict.lm(mod, interval = "prediction"): predictions on current data refer to
    _future_ responses
```

```
RailTrail %>%
  mutate(lower.pi = pi[, "lwr"], upper.pi = pi[, "upr"]) %>%
  ggplot(aes(x = hightemp, y = volume)) +
  geom_point() +
  geom_smooth(method = "lm", formula = y ~ x, se = TRUE) +
  geom_line(aes(y = lower.pi), colour = "red", linetype = "dashed") +
  geom_line(aes(y = upper.pi), colour = "red", linetype = "dashed")
```




**The two interval formulas [Supplement]**

With $S_{xx} = \sum(x_i - \bar{x})^2$ and $s^2 = \text{SSE}/(n-2)$, the intervals at $x_0$ are

$$\hat{y}_0 \pm t_{n-2,0.975}\, s\sqrt{\frac{1}{n} + \frac{(x_0-\bar{x})^2}{S_{xx}}} \quad \text{(mean)}, \qquad \hat{y}_0 \pm t_{n-2,0.975}\, s\sqrt{1 + \frac{1}{n} + \frac{(x_0-\bar{x})^2}{S_{xx}}} \quad \text{(new obs.)}.$$

The extra 1 under the second root is the variance of the new error; both widen away from $\bar{x}$.

### 5.1.3 Strength of fit and least squares

With two variables the correlation $r$ measures the strength of the linear relationship; in general we use

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \in [0,1],$$

the fraction of variation explained, comparing the fitted model with the null model $y_i = \beta_0 + \epsilon_i$. Since SST is fixed, a smaller SSE is a better fit, and least squares finds the line with the smallest SSE:

$$\sum_i \{y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)\}^2 \le \sum_i \{y_i - (\beta_0 + \beta_1 x_i)\}^2 \quad \text{for all } \beta_0, \beta_1.$$

Least squares became standard partly for computational convenience, which still matters for big data; like the mean, it is sensitive to outliers, mainly a problem in small or moderate samples. More generally, *any statistical method optimises a goodness-of-fit measure over a family of models*: sometimes the optimum has a formula, often it must be approximated by an algorithm, and comparing several families makes computational efficiency important.

### 5.1.4 A categorical predictor

With the indicator `weekday` (1 on weekdays, 0 on weekends and holidays) the model $\text{volume}_i = \beta_0 + \beta_1 \text{weekday}_i + \epsilon_i$ has only two fitted values, the two group means:

```
RailTrail %>% group_by(weekday) %>% summarise(mean_volume = mean(volume))
```

```
## # A tibble: 2 x 2
##   weekday mean_volume
##   <lgl>         <dbl>
## 1 FALSE          431.
## 2 TRUE           350.
```

```
coef(lm(volume ~ weekday, data = RailTrail))
```

```
## (Intercept) weekdayTRUE
##   430.71429   -80.29493
```

About 80 fewer riders are expected on a weekday. `lm()` takes the alphabetically first level as the **reference group** and creates indicators for the others; recoding changes the reference and flips the sign:

```
RailTrail <- RailTrail %>%
  mutate(day = ifelse(weekday == 1, "weekday", "weekend/holiday"))
coef(lm(volume ~ day, data = RailTrail))
```

```
##        (Intercept) dayweekend/holiday
##          350.41935           80.29493
```

## 5.2 Loss functions

**Motivation.** "Best fit" needs a definition. Choosing the loss function defines it, and different losses give different answers, most visibly when the data contain outliers.

A **loss function** $L(y, \hat{y})$ measures the cost of predicting $\hat{y}$ when the truth is $y$. The two most common are the squared ($L_2$) loss $(y-\hat{y})^2$ and the absolute ($L_1$) loss $|y-\hat{y}|$. Both are large when $\hat{y}$ is far from $y$, but $L_2$ penalises large residuals much more.

The model is fitted by minimising the average loss over the data, the **empirical risk**

$$\hat{R}(\beta_0, \beta_1) = \frac{1}{n}\sum_i L(y_i, \hat{y}_i),$$

which measures fit to the sample, not the population. With $L_2$ it is the mean squared error (MSE), and with $L_1$ the mean absolute error (MAE). Least squares is exactly minimisation of the $L_2$ risk. Setting the derivatives to zero gives the estimating equations $\frac{1}{n}\sum_i(y_i-\hat{y}_i)=0$ and $\frac{1}{n}\sum_i(y_i-\hat{y}_i)x_i=0$, whose solution is

$$\hat{\beta}_1 = \frac{\sum_i (x_i-\bar{x})(y_i-\bar{y})}{\sum_i (x_i-\bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

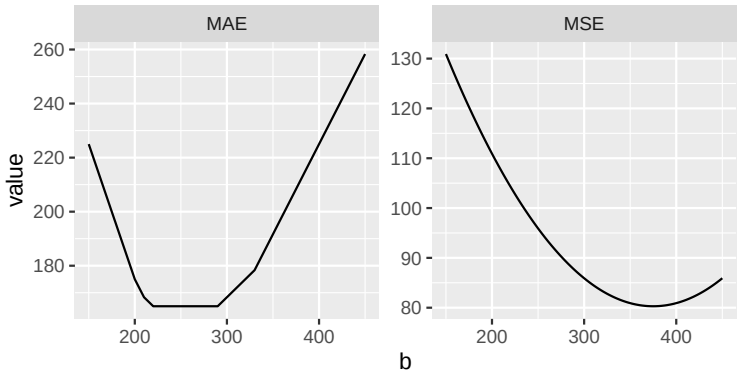
**Proposition 5.1 (constant model)**

For the constant model $y = \beta_0 + \epsilon$, the minimiser of the $L_2$ risk is $\bar{y}$ and a minimiser of the $L_1$ risk is any median of $y_1, \dots, y_n$.

**Proof.** $L_2$: $\frac{d}{d\beta_0}\frac{1}{n}\sum(y_i-\beta_0)^2 = -\frac{2}{n}\sum(y_i-\beta_0) = 0$ gives $\beta_0 = \bar{y}$, and the risk is a convex parabola. $L_1$: away from the data points, $\frac{d}{d\beta_0}\frac{1}{n}\sum|y_i-\beta_0| = \frac{1}{n}(\#\{y_i<\beta_0\} - \#\{y_i>\beta_0\})$. The risk is convex and piecewise linear, decreasing while fewer than half the points lie below $\beta_0$ and increasing once more than half do, so it is minimised where the numbers of points below and above balance, i.e. at a median. ■

**Example (bubble-tea sales).** Weekday sales on five random days are 200, 210, 220, 290, 330. A constant model predicts 250 (mean, $L_2$) or 220 (median, $L_1$). Adding an outlying day of 1000 moves the mean to 375 but the median only to 255; and with six points the $L_1$ minimiser is not unique: any value between 220 and 290 minimises the MAE.

```
y <- c(200, 210, 220, 290, 330, 1000)
tibble(b = seq(150, 450, by = 1)) %>%
  mutate(MSE = map_dbl(b, ~ mean((y - .x)^2)) / 1000,
         MAE = map_dbl(b, ~ mean(abs(y - .x)))) %>%
  pivot_longer(-b) %>%
  ggplot(aes(b, value)) + geom_line() + facet_wrap(~ name, scales = "free_y")
```




MSE is smooth (easy to minimise) with a unique minimum at the mean but is sensitive to outliers; MAE is piecewise linear (harder to minimise), flat between 220 and 290, and robust. (MSE is shown in thousands.)

## 5.3 Multiple regression

**Motivation.** Ridership depends on temperature, rain and the day of the week at the same time. Multiple regression estimates each effect while holding the others fixed, which is what makes its coefficients meaningful but also subtle.

The model is $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \epsilon$. Each $\beta_j$ is the predicted change in $y$ for a one-unit increase in $x_j$ *holding the other predictors fixed.*

```
mod_plane <- lm(volume ~ hightemp + precip, data = RailTrail)
tidy(mod_plane)
```

```
## # A tibble: 3 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)   -31.5     55.2      -0.571 5.70e- 1
## 2 hightemp        6.12     0.794     7.70  1.97e-11
## 3 precip       -153.      39.3      -3.90  1.90e- 4
```

An inch of rain lowers predicted ridership by about 153 at a given temperature. Three questions guide multiple regression: how uncertain are the coefficients, which predictors are useful, and how accurate are predictions?

### 5.3.1 Inference for coefficients

Uncertainty arises in predicting $y$ from $x$ and in estimating the coefficients. The 95% confidence interval by normal approximation is $\hat{\beta}_j \pm 1.96\,\mathrm{SE}(\hat{\beta}_j)$; `confint()` uses $t$ quantiles:

```
mod_p_planes <- lm(volume ~ hightemp + precip + weekday, data = RailTrail)
tidy(mod_p_planes)
```

```
## # A tibble: 4 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)    19.3     60.3       0.320 7.50e- 1
## 2 hightemp        5.80     0.799     7.26  1.59e-10
## 3 precip       -146.      38.9      -3.74  3.27e- 4
## 4 weekdayTRUE   -43.1     22.2      -1.94  5.52e- 2
```

```
confint(mod_p_planes)
```

```
##                    2.5 %      97.5 %
## (Intercept) -100.630740 139.2683959
```

```
## hightemp       4.213254   7.3880945
## precip      -222.927105 -68.2908651
## weekdayTRUE  -87.265016   0.9763902
```

Without the population we can still use the bootstrap (Lecture 4): resample the days, refit, and take the standard deviation of the coefficients.

```
set.seed(1)
n <- nrow(RailTrail)
coef_fun <- function(data) lm(volume ~ hightemp + precip + weekday, data = data)$coef
boot <- 1:500 %>%
  map_dfr(~ RailTrail %>% slice_sample(n = n, replace = TRUE) %>% coef_fun())
round(coef(mod_p_planes)["hightemp"] + c(-1.96, 1.96) * sd(boot$hightemp), 2)
```

```
## [1] 4.2 7.4
```

The bootstrap interval for `hightemp` is close to the model-based one.

### 5.3.2 Choosing a model

Adding variables always increases $R^2$, but a bigger model is not necessarily better. **Occam's razor**: other things equal, prefer the simpler model. Candidate models, from small to full (`volume ~ .` uses all variables):

```
mod1 <- lm(volume ~ hightemp + weekday, data = RailTrail)
mod2 <- lm(volume ~ hightemp + cloudcover + weekday, data = RailTrail)
mod3 <- lm(volume ~ . - day, data = RailTrail)
map_dfr(list(mod1 = mod1, mod2 = mod2, mod3 = mod3), glance, .id = "model") %>%
  select(model, r.squared, adj.r.squared, AIC, BIC)
```

```
## # A tibble: 3 x 5
##   model r.squared adj.r.squared   AIC   BIC
##   <chr>     <dbl>         <dbl> <dbl> <dbl>
## 1 mod1      0.374         0.359 1093. 1103.
## 2 mod2      0.459         0.440 1082. 1094.
## 3 mod3      0.511         0.469 1081. 1103.
```

$R^2$ grows with every added variable, but BIC prefers `mod2`, while adjusted $R^2$ and AIC slightly favour the full model: the criteria trade fit against size differently. Criteria that penalise size:

- adjusted $R^2_{\text{adj}} = 1 - (1 - R^2)\dfrac{n-1}{n-p-1}$, with $p$ the number of predictors;
- AIC (Akaike's information criterion) and BIC (Bayesian, with a stronger penalty on $p$), smaller is better;
- Mallows' $C_p$.

Searching all $2^p$ subsets (**all-subset regression**) is expensive; **stepwise regression** instead drops variables from the full model (backward), adds them to the constant model (forward), or does both (`MASS::stepAIC()`). **Penalized regression** (ridge, lasso; Lecture 7) shrinks coefficients instead of removing variables.

These are in-sample methods, so model selection can **overfit** (fit the noise) and perform worse on new data; cross-validation (Lecture 6) guards against this. For linear regression overfitting is rarely severe because of the simple global structure, but for flexible, locally adaptive methods cross-validation is essential.

## 5.4 Other considerations

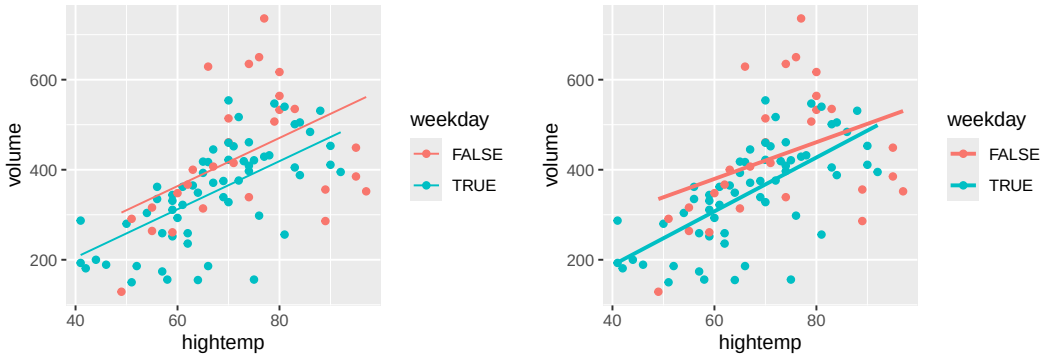
**Factor variables** are coded by dummy (indicator) variables: *reference coding* drops one level, *one-hot encoding* keeps all (and then drops the intercept). Ordered categories may be treated as numeric scores.

**Interactions.** The linear model assumes the effects are *additive* (the effect of $x_1$ does not depend on $x_2$) and *linear*. With a 0/1 variable $x_2$ (weekday), $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2$ gives parallel lines: intercept $\hat{\beta}_0$ on weekends and $\hat{\beta}_0 + \hat{\beta}_2$ on weekdays, same slope $\hat{\beta}_1$. Adding $x_1 x_2$,

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \hat{\beta}_3 x_1 x_2,$$

gives slope $\hat{\beta}_1$ on weekends and $\hat{\beta}_1 + \hat{\beta}_3$ on weekdays: an **interaction model**.

```
ggplot(RailTrail, aes(hightemp, volume, colour = weekday)) +
  geom_point() +
  geom_line(aes(y = fitted(lm(volume ~ hightemp + weekday, data = RailTrail))))
ggplot(RailTrail, aes(hightemp, volume, colour = weekday)) +
  geom_point() + geom_smooth(method = "lm", formula = y ~ x, se = FALSE)
```




Left: parallel slopes; right: separate slopes (interaction).

**Nonlinear relationships.** Height against age for females in `NHANES` is clearly not linear: a straight line misfits badly, while a LOESS smoother follows the growth curve. The better fit has a price: the smoother's confidence band is wider. Polynomial regression, splines, local regression and GAMs are studied in Lecture 10.

```
NHANES %>% filter(Gender == "female", !is.na(Height)) %>%
  slice_sample(n = 1500) %>%
  ggplot(aes(Age, Height)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "lm", formula = y ~ x, colour = "blue") +
  geom_smooth(method = "loess", formula = y ~ x, colour = "green4")
```

